GENERATIVE ARTIFICIAL INTELLIGENCE · MODULE 1

Quiz Answer Key & Detailed Explanations

20 depth-reasoning questions — full worked solutions, for instructor / post-quiz distribution.

Dr. Chinmaya Mishra · IIIT Sri City | Covers: Symbolic & Statistical AI · ML Foundations · Neural Networks & Backprop · RNN/LSTM/GRU · Attention · Transformers · LLM Architecture · Fine-Tuning/LoRA

Instructor note: Each explanation below goes beyond the one-line rationale used in the quiz itself — it derives the answer from the underlying formula/mechanism and explains concretely why each distractor is wrong, so this document can stand alone as a study resource after the quiz closes.

Q1 — SYMBOLIC VS. STATISTICAL AI

A hospital wants a diagnostic system explainable enough to survive a legal audit, but must still degrade gracefully when symptoms are ambiguous or missing. Which design best satisfies both constraints, and why?

A A pure symbolic rule-based system, since every conclusion traces back to a rule

B A pure black-box neural classifier, since it handles noisy input well

C A rule-based system where each rule's firing carries a Bayesian confidence score ✓ CORRECT

D Discard the rules entirely and train a purely statistical model from scratch

EXPLANATION

Symbolic AI's strength is full traceability — every conclusion follows deductively from an explicit, human-readable rule, which is exactly what an audit needs. But pure symbolic systems are brittle: a rule either fires or it doesn't, with no graceful way to handle a partial or ambiguous match. Pure statistical/black-box models handle noise naturally but cannot produce a step-by-step justification for a specific decision, failing the audit requirement outright. Attaching a Bayesian confidence score to each rule's firing is the actual historical resolution the module traces: keep the explicit rule structure (so every conclusion still traces to a named rule) while letting the system express graded belief when evidence is incomplete. **D** abandons interpretability entirely; **A** and **B** each satisfy only one of the two stated requirements.

Q2 — BAYESIAN REASONING

A screening test has base rate P(D) = 2%, sensitivity P(+|D) = 90%, and false-positive rate P(+|¬D) = 8%. A random person tests positive. What is P(D|+), closest to?

A 90%

B 50%

C ≈ 19% ✓ CORRECT

D ≈ 2%

EXPLANATION

By the law of total probability: P(+) = P(+|D)P(D) + P(+|¬D)P(¬D) = 0.90×0.02 + 0.08×0.98 = 0.018 + 0.0784 = 0.0964. Then by Bayes' theorem: P(D|+) = P(+|D)P(D)/P(+) = 0.018/0.0964 ≈ 0.187, i.e. ≈18.7%, closest to **C**. The intuitive-but-wrong answer (**A**, 90%) conflates the test's sensitivity P(+|D) with the posterior P(D|+) — the classic base-rate fallacy. Because the disease is rare, the healthy population (98%) is so much larger than the sick population (2%) that even an 8% false-positive rate among healthy people (0.0784) generates more false positives in absolute terms than the true positives from the sick group (0.018), pulling the posterior well below 90%. This is the identical mechanism as the professor's own rare-disease worked example (16.1% posterior) — same formula, different input numbers, testing whether the mechanism (not a memorized number) was understood.

Q3

LINEAR REGRESSION — FOUR VIEWS

Which statement about the matrix, Newton, gradient-descent, and Bayesian derivations of linear regression is correct?

A They converge to different optimal β because each uses a different loss function

B Newton's method reaches the exact minimum in one step for squared-error loss, because the cost surface is exactly quadratic ✓ CORRECT

C Gradient descent is preferred over the normal equation on small datasets because it is more numerically accurate

D The Bayesian derivation requires no assumption about the noise distribution

EXPLANATION

All four methods minimize the identical convex objective $J(\beta) = \|y - X\beta\|^2$, so they converge to the same $\beta = (X^TX)^{-1}X^Ty$ — **A** is false; there is no divergence, because there is no difference in loss. Newton's update uses both the gradient (slope) and the Hessian (curvature): for squared-error loss the Hessian X^TX is *constant* everywhere, i.e. the cost surface is a perfect quadratic bowl rather than something only locally approximated by a quadratic — so a single Newton step lands exactly on the minimum, with no iteration required (this is special to quadratic losses; for something like logistic regression's cross-entropy loss, Newton must iterate). Gradient descent reaches the same minimum only after many small steps — its advantage is scalability to huge models, not accuracy, so **C** is backwards (for a tiny dataset the closed-form normal equation is if anything the more exact solve). The Bayesian derivation explicitly assumes Gaussian noise $\varepsilon \sim N(0, \sigma^2)$ and a prior distribution over β — it doesn't remove assumptions, it makes them explicit, so **D** is false.

Q4

MODEL SELECTION — D ≫ N

You have 300 rows and 5,000 sparse one-hot categorical features (d ≫ n). What is the most defensible first model, and why?

A A deep neural network — more parameters means more capacity for sparse data

B Unregularized linear regression — more features means more information

C Ridge or Lasso regression — regularization controls overfitting when features vastly outnumber rows ✓ CORRECT

D A single unpruned decision tree — trees need no feature scaling

EXPLANATION

When the number of features (5,000) vastly exceeds the number of rows (300), an unregularized linear model has more free parameters than data points — infinitely many β values satisfy the normal equations exactly, and OLS has no principled way to pick among them, so it will overfit catastrophically (**B** fails for exactly this reason, despite sounding intuitive). Ridge (L2) and Lasso (L1) add a penalty term that shrinks or zeroes out coefficients, directly controlling this degenerate over-parameterization — this is precisely the guidance in the "d ≫ n" row of the model-selection table. A deep neural network (**A**) only compounds the over-parameterization problem and needs vastly more than 300 rows to avoid memorizing the training set. A single unpruned decision tree (**D**) also overfits badly at this row-to-feature ratio and offers no analogous regularization mechanism for high-dimensional sparse inputs.

Q5

GENERATIVE VS. DISCRIMINATIVE

A fraud-detection team has very few labeled fraud examples but abundant unlabeled transaction data. Why might a generative model (e.g., Naive Bayes) outperform a purely discriminative classifier here?

A Generative models always have strictly lower bias than discriminative models

B By modeling the joint P(x,y) with explicit distributional assumptions, generative models can estimate parameters reliably from far fewer labeled examples ✓ CORRECT

C Discriminative models cannot produce probability outputs

D Generative models require no labeled data at all to function

EXPLANATION

Naive Bayes doesn't just learn a decision boundary — it estimates the full class-conditional distributions P(x|y) and the class prior P(y), under the simplifying assumption that features are conditionally independent given the class. That strong assumption means each per-feature distribution can be estimated reliably from far fewer labeled examples than a discriminative model needs to directly learn a flexible P(y|x) boundary from scratch — this is exactly why Naive Bayes is the textbook "first baseline" when labels are scarce. **A** overclaims: the independence assumption is itself a strong (sometimes overly high-bias) assumption, not a universal bias advantage. **C** is simply false — logistic regression, a discriminative model, outputs calibrated probabilities directly. **D** is false — Naive Bayes still requires labeled data to estimate P(y) and the class-conditional parameters; it is not unsupervised.

Q6

VANISHING GRADIENTS

A 20-layer network uses sigmoid activations throughout. The first 5 layers barely update during training while the last few layers train normally. What's the most likely cause and best fix?

A Learning rate is too high — halve it

B Vanishing gradients from repeated multiplication by sigmoid derivatives (max 0.25 each); switch hidden layers to ReLU or add residual connections ✓ CORRECT

C The loss function is wrong — switch cross-entropy to MSE

D The network needs more layers so gradients have room to flow

EXPLANATION

Sigmoid's derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$ peaks at exactly 0.25 (at z=0) and shrinks toward 0 elsewhere. Backpropagation multiplies local derivatives along the entire chain from the output back to a given layer, so through even a modest stack the product shrinks geometrically: through 10 sigmoid layers, best case is $0.25^{10} \approx 0.00000095$ — essentially zero. Gradients reaching the earliest layers vanish, so those layers stop updating, while later layers (which the gradient reaches after fewer multiplications) train fine — exactly the symptom described. The fix follows the diagnosis directly: ReLU's derivative is a constant 1 for all positive inputs (no shrinkage), and residual/skip connections give the gradient an additive bypass around the multiplicative chain entirely. A high learning rate (**A**) would cause oscillation or divergence across all layers, not a selective early-layer freeze; the wrong loss (**C**) wouldn't produce this specific depth-dependent pattern; and adding more layers (**D**) only lengthens the multiplicative chain, making vanishing gradients strictly worse.

Q7

ADAM OPTIMIZER

Why does a parameter that has historically received large, frequent gradients get a smaller effective step size under Adam than a parameter with small, infrequent gradients?

A Adam clips every gradient to a fixed maximum norm

B Adam divides the step by a running estimate of the root-mean-square of past squared gradients, so parameters with a history of large gradients get automatically down-scaled ✓ CORRECT

C Adam uses second-order Hessian curvature like Newton's method

D Adam multiplies the gradient by the current loss value

EXPLANATION

Adam tracks two running averages per parameter: v (a decaying average of the gradient itself — momentum) and s (a decaying average of the *squared* gradient, borrowed from RMSProp: $s \leftarrow \beta_2 s + (1-\beta_2)g^2$). The update divides the bias-corrected momentum by $\sqrt{\hat{s}} + \varepsilon$. A parameter with a history of large gradients accumulates a large s, so dividing by the large $\sqrt{\hat{s}}$ automatically shrinks its effective step — Adam is "de-risking" parameters that have already been moving a lot. A parameter with small/infrequent gradients keeps a small s, so its relatively larger effective step keeps it making progress. This has nothing to do with fixed gradient clipping (**A**), Hessian curvature (**C**, that's Newton's method, not Adam), or scaling by the loss value (**D**, which isn't part of the Adam update at all).

Q8

LSTM GRADIENT FLOW

Using the cell-state recurrence, why does an LSTM whose forget gate has saturated near 1 preserve gradient signal across many timesteps, while a vanilla RNN cannot do the analogous thing?

A Because tanh outputs are always close to 1

B Because $\partial c_t/\partial c_{t-1} = f_t \approx 1$ makes the cell-state gradient path near-identity/additive, whereas a vanilla RNN's $\partial h_t/\partial h_{t-1}$ always contains a multiplicative $W_{hh}\cdot\tanh'$ term that is generally < 1 ✓ CORRECT

C Because LSTMs have more parameters and therefore always learn faster

D Because LSTMs don't use backpropagation through time

EXPLANATION

The cell-state recurrence is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ — an *addition*, not a matrix multiplication. Differentiating gives $\partial c_t/\partial c_{t-1} = f_t$ directly; if the forget gate sits near 1, gradient flowing backward along the cell-state path passes through almost unchanged, timestep after timestep, regardless of how many steps back you go. Contrast a vanilla RNN's $h_t = \tanh(W_{hh} h_{t-1} + \ldots)$: here $\partial h_t/\partial h_{t-1} = W_{hh}^{\top}\cdot\mathrm{diag}(\tanh'(z_t))$ is a genuine matrix multiplication at *every single step*, and since $\tanh'(z) \le 1$ always and W_{hh} is generally not the identity, this term is typically < 1 in magnitude — so the gradient shrinks geometrically over many timesteps, exactly like Q6's sigmoid case. It's the additive-versus-multiplicative structure of the recurrence itself that matters — not merely "having gates" in some vague sense (**A**), and certainly not parameter count (**C**) or the (false) claim that LSTMs skip BPTT (**D**).

Q9

GRU VS. LSTM

A GRU's update gate couples "keep old state" and "write new content" so that (1−z) and z must sum to 1. What concrete capability does this cost GRU relative to LSTM?

A GRU cannot process sequences at all

B GRU cannot simultaneously retain most of the old state AND write a large amount of new information at the same timestep — increasing z to write more automatically forces less of the old state to be kept ✓ CORRECT

C GRU always trains slower than LSTM

D GRU has no hidden state whatsoever

EXPLANATION

GRU's update is $h_t = (1-z_t)\odot h_{t-1} + z_t \odot \tilde{h}_t$ — the coefficients on the old state and the new candidate are forced to sum to exactly 1 by construction, not independently controllable. If the model wants to write a lot of new information (large z_t), it is mathematically forced to discard a correspondingly large fraction of the old state — it cannot do both "keep almost everything" and "write a lot" at once. LSTM's forget gate f_t and input gate i_t are two entirely separate, independently-learned sigmoids in $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, with no such coupling — the model can set f_t near 1 (retain nearly all old memory) *and* i_t large (write substantial new information) simultaneously. This is the concrete expressiveness gap, purchased by LSTM's extra gate (and ~33% more parameters); GRU is in fact generally *faster* to train (**C** is backwards), not slower.

Q10

BAHDANAU ATTENTION

In Bahdanau attention, H (the encoder states) is computed once and frozen. Why must α be recomputed at every decoder timestep even though H never changes?

A Because H actually does change at every timestep

B Because the raw score e_{t,i} depends on the decoder's current state s_{t−1}, which changes every step, so the blending recipe over the same fixed H must be re-derived each time ✓ CORRECT

C Because softmax can only be applied once per sequence

D Because α is itself a learned parameter matrix, not a computed value

EXPLANATION

The scoring function $e_{t,i} = v_a^T \tanh(W_a s_{t-1} + U_a h_i + b_a)$ takes two inputs: the fixed encoder state h_i and the decoder's current state s_{t-1}. h_i never changes after encoding finishes, but s_{t-1} is a brand-new vector after every decoder step (it encodes "what I've generated so far") — so the score, and therefore the softmax-normalized α, must be recomputed from scratch every timestep even though the "raw material" H being scored against is unchanged (the "smoothie" analogy: the fruit never changes, but the recipe is re-decided for every glass). **D** targets the single most common point of confusion the study guide explicitly flags: the word "weight" is overloaded — W_a, U_a, v_a, b_a are the permanently *learned* parameters (trained once via backprop, fixed at inference), while α is never learned or stored at all; it is a fresh numerical output of scoring + softmax, discarded and recomputed every step.

Q11 BAHDANAU VS. LUONG

What is the essential mathematical difference between Bahdanau (additive) and Luong dot-product attention, and what constraint does Luong-dot impose that Bahdanau does not?

A Bahdanau uses softmax; Luong does not

B Bahdanau projects query and key, adds them, and collapses via tanh + a learned vector v_a; Luong dot uses a raw dot product with no learned scoring parameters, requiring matching query/key dimensionality ✓ CORRECT

C Luong attention cannot be trained end-to-end with backpropagation

D Bahdanau only works with a bidirectional encoder

EXPLANATION

Bahdanau's score $v_a^T \tanh(W_a s + U_a h_i + b_a)$ projects the decoder state and encoder state into a shared space with two separate learned matrices, *adds* the two projections, passes the sum through a nonlinearity, and collapses to a scalar with a learned vector — this "add, then nonlinear collapse" is why it's named *additive* attention. Luong's simplest ("dot") variant instead computes a raw dot product $s_t^T h_i$ directly, with no learned projection matrices at all. Because a dot product is only defined between vectors of matching dimensionality, Luong-dot forces the decoder and encoder hidden sizes to be equal — a constraint Bahdanau's learned projections (which can map two *different*-sized spaces into a shared comparison space) simply doesn't have. (Luong's "general" variant, $s_t^T W_a h_i$, restores this flexibility with a single learned matrix, as a middle ground between the two extremes.) Both

mechanisms remain fully differentiable and end-to-end trainable, so **C** is false, and neither requires a bidirectional encoder specifically (**D** is false — bidirectionality helps H be a richer representation, but is a property of the encoder, not the scoring function).

Q12

SCALED DOT-PRODUCT ATTENTION

Explain, in terms of softmax's behavior, why unscaled dot-product attention (QK^T without dividing by $\sqrt{d_k}$) becomes harder to train as d_k grows.

A Larger d_k makes matrix multiplication itself numerically unstable regardless of value magnitude

B As d_k grows, dot products of random vectors have larger variance, pushing some raw scores far apart; softmax then saturates toward near one-hot outputs, and softmax's gradient near saturation is close to zero, killing the training signal ✓ CORRECT

C Larger d_k makes the attention matrix non-square, breaking softmax

D It doesn't matter — the $\sqrt{d_k}$ division is a stylistic convention with no effect on training

EXPLANATION

Each entry of QK^T is a dot product over d_k dimensions; if the components of Q and K have roughly fixed per-dimension variance, the variance of the resulting dot product grows *linearly* with d_k (you're summing d_k roughly-independent products). So as d_k grows, raw attention scores spread further apart in magnitude purely as an artifact of dimensionality, not because tokens are genuinely more or less related. Feeding a widely-spread set of scores into softmax causes saturation: softmax pushes nearly all probability mass onto whichever score happens to be largest, producing something close to a one-hot vector. But softmax's gradient — proportional to terms like p(1–p) — is close to zero exactly in that saturated regime, so backpropagation through a saturated softmax carries almost no learning signal, and the alignment network effectively stops improving. Dividing every score by $\sqrt{d_k}$ exactly cancels the linear-in-d_k variance growth, keeping the score spread roughly constant regardless of dimension and keeping softmax in its more sensitive, better-gradiented middle range. This is a real training-dynamics effect (falsifying **D**), not a numerical-stability or shape issue (**A**, **C**).

Q13

MULTI-HEAD ATTENTION

Why is running 8 attention heads of dimension 64 each roughly the same FLOP cost as running 1 head of dimension 512 — and what is gained "for free" by splitting?

A Multi-head is NOT equivalent in cost — it is always 8× more expensive than a single head

B Total projected dimensionality is unchanged (8×64 = 512), so projection FLOPs match; what's gained is multiple independent subspaces, each free to specialize in a different relational pattern instead of one softmax-averaged compromise ✓ CORRECT

C Splitting into heads removes the need for positional encoding entirely

D Each head receives an entirely separate input sequence to process

EXPLANATION

d_model=512 split across h=8 heads gives each head d_k=64 (512/8=64) — the total number of dimensions being projected across all heads (8×64=512) is identical to projecting directly at 512 dimensions with one head, so the aggregate Q/K/V projection FLOPs match either way; splitting partitions the same width rather than adding to it, so **A** is false. What's actually gained: a single attention head produces one softmax-weighted average, which necessarily *destroys* information by forcing a compromise across everything a token might need to attend to (its syntactic subject, a distant coreference, positional neighbors, etc., all blended into one sum). Multiple heads let each subspace specialize independently — empirically, trained Transformers really do develop heads that separately track position, syntax, or coreference, exactly as the guide's table of observed head behaviors documents. You effectively get several independent "relational views" of the sequence at the same total compute, which a single big head cannot provide because averaging structurally erases the distinctions between those separate patterns. Positional encoding (**C**) and per-head separate inputs (**D**) are unrelated to this mechanism.

Q14

CAUSAL MASKING

A decoder-only Transformer processes an entire target sequence in parallel during training (with a causal mask) but generates one token at a time at inference. Why does the causal mask make this split consistent, and what would go wrong without it?

A Without masking, softmax outputs would no longer sum to 1

B Without the mask, position i could attend to future ground-truth tokens fed in during parallel teacher-forced training — the model would learn to copy the answer rather than predict it, a shortcut that doesn't exist at inference ✓ CORRECT

C Without masking, the model could no longer use a KV cache

D Masking is only relevant for encoder-only architectures, not decoders

EXPLANATION

During training, the whole target sequence is fed to the decoder at once via teacher forcing — the true token at position i+1 sits right there in the same input batch as position i. Without a mask, self-attention would let position i's query attend directly to position i+1's key/value, letting the model trivially "cheat" by copying the true next token out of its own input rather than learning to predict it from prior context. The causal mask (setting all j>i score entries to −∞ before softmax) makes this structurally impossible — position i mathematically sees only itself and earlier positions — so the only way to achieve low loss is to genuinely learn next-token prediction from context, which is exactly the situation the model is actually in at inference (where token i+1 doesn't exist yet). This alignment between what's visible during training and what's available at inference is what makes parallel-training-but-sequential-generation coherent rather than a train/inference mismatch. It's unrelated to softmax's sum-to-1 property (**A**, unaffected by masking), orthogonal to KV-caching (**C**, a separate inference-time optimization), and it is decoder *self*-attention specifically that needs the mask — encoders are deliberately bidirectional, making **D** backwards.

Q15

ROPE

Using RoPE with θ = 15° per position step and toy 2D content vectors (1,0) at every position, two tokens sit at positions 3 and 7. What is their positional attention-score

contribution, and what property does this illustrate?

A cos(10°) ≈ 0.985 — showing RoPE depends on absolute position

B cos(60°) = 0.5 — showing the score depends only on the relative gap between positions (7–3 = 4 steps × 15° = 60°), never on absolute positions ✓ CORRECT

C sin(60°) ≈ 0.866 — showing RoPE always uses the sine component

D cos(15°) — because only the smaller of the two positions matters

EXPLANATION

RoPE's proven identity is q(pos_i)·k(pos_j) = cos((pos_i – pos_j)·θ) for the toy content vector (1,0) rotated to angle ϕ = pos·θ — depending *only* on the gap i–j, never on i or j individually. Here the gap between positions 7 and 3 is 4 steps, giving angle 4×15° = 60°, so the score is cos(60°) = 0.5. This is the entire practical payoff of RoPE: the identical relative-position score would result whether the two tokens sat at (3,7) or, say, (103,107) — anywhere with the same 4-step gap — which is exactly what lets RoPE generalize positional reasoning across absolute positions and extrapolate better to longer sequences than absolute schemes. **A** applies the formula to the wrong gap; **C** incorrectly generalizes — the sine term vanishes specifically because this toy vector is (1,0), not as a general RoPE property; **D** is wrong because it's the *difference* between positions that matters, not either position alone.

Q16

TOKENIZATION

A BPE-tokenized model is asked "how many letters are in strawberry?" and gets it wrong. What is the mechanistic reason (not just "LLMs are bad at counting")?

A The model has simply never encountered the word "strawberry" during training

B BPE most likely tokenizes "strawberry" as one or two subword tokens rather than ten separate characters, so the model's internal representation never exposes individual letters to count — it only reasons over opaque token IDs ✓ CORRECT

C The model does not understand the English language

D Counting requires an external calculator tool that token-based models inherently lack

EXPLANATION

Byte-Pair Encoding builds its vocabulary by greedily merging the most frequent adjacent character pairs, repeated until a target vocabulary size is reached. A common English word like "strawberry" is very likely represented as one whole token, or at most a couple of subword pieces (e.g., "straw"+"berry"), rather than as its ten individual letters. Since self-attention and every downstream layer operate purely on token IDs/embeddings — never on raw characters — the model has no direct access to the letters s-t-r-a-w-b-e-r-r-y to count; it can only reason about the token(s) as opaque units. This makes the failure *architectural*, not a knowledge gap: the model almost certainly saw "strawberry" thousands of times during training (**A** is false), the failure has nothing to do with language comprehension broadly (**C**), and while tool-use can mitigate arithmetic problems, it doesn't explain the underlying representational cause here (**D**).

Q17

KV CACHE

Generating 500 new tokens one at a time WITHOUT a KV cache costs how much more, roughly, than with one — and why does this matter operationally?

A The same either way — caching only affects memory use, not compute

B Without caching, K/V for all prior tokens is recomputed at every new step, so total compute grows roughly $O(n^2)$ in tokens generated; with caching it drops to $O(n)$ — this is why latency/cost scale so much better with a cache, especially in long conversations ✓ CORRECT

C Without a cache the model produces different, incorrect outputs

D KV caching only matters during training, never during inference

EXPLANATION

Without a cache, generating the n-th token requires recomputing K and V for all n−1 previous tokens from scratch, and this repeats at every one of the n generation steps — total work across all steps is proportional to $1+2+\ldots+n \approx n^2/2$, i.e. $O(n^2)$. With a cache, each step computes K/V for only the one new token and appends it to what's already stored, so total work is proportional to n, i.e. $O(n)$. At n=500 this is the difference between roughly 500 units and roughly 125,000 units of repeated computation — not a small constant-factor gap. Operationally, this is exactly why a long chat conversation feels progressively slower and consumes more GPU memory: the cache itself must be held for the whole conversation, and each additional generated token both grows the cache and, without caching, would make every subsequent step increasingly expensive. Caching is purely an inference-time optimization (training already processes the full sequence in parallel, so **D** is backwards) and doesn't change what the model computes, only how much redundant work is avoided, so **C** is false.

Q18

TOP-K VS. TOP-P SAMPLING

For one prompt the next-token distribution is sharply peaked (one token at 92%); for another it's nearly flat. Comparing top-k=5 against top-p=0.9 on both, which behaves more "appropriately" across the two cases, and why?

A Top-k, because it always removes exactly the same number of low-probability tail tokens

B Top-p, because it adapts the number of retained candidates to the shape of the distribution — very few when confident, more when uncertain — whereas top-k always keeps a fixed count regardless of confidence ✓ CORRECT

C Neither differs meaningfully; the two methods are mathematically identical

D Top-k, because probability mass carries no information about token quality

EXPLANATION

Top-k always keeps exactly k candidates: on the sharply peaked distribution, top-k=5 artificially retains 4 essentially-irrelevant low-probability tokens even though the model is extremely confident; on the flat distribution it might arbitrarily exclude a 6th token nearly as good as the 5th. Top-p instead keeps however many tokens are needed to reach a target *cumulative* probability mass (here 90%): on the peaked distribution this might take just one or two tokens, correctly reflecting confidence by barely allowing alternatives; on the flat distribution reaching 90% may require many more tokens, correctly reflecting genuine uncertainty by keeping more options open. This adaptivity to the actual shape of the

distribution — rather than a fixed count — is why production APIs typically default to tuning top-p and/or temperature rather than top-k.

Q19

HALLUCINATION & TEMPERATURE

A developer sets temperature = 0 (fully greedy decoding), believing this will eliminate hallucination. Why is this reasoning flawed?

A Temperature = 0 is not actually implementable in real systems

B Temperature/top-p/top-k only control how the model samples among tokens it already finds plausible — they cannot fix wrong or missing knowledge in the model's weights; a confidently wrong top-ranked token will be selected every single time under greedy decoding ✓ CORRECT

C Temperature = 0 always increases hallucination because it removes beneficial randomness

D Hallucination can only be reduced by increasing model size, never decoding parameters

EXPLANATION

Every decoding step — logits → divide by temperature → softmax → optional top-k/top-p filtering → sample — only ever reshapes or restricts the distribution over tokens the model already considers plausible, based on what it learned during pretraining. None of these steps can inject knowledge the model never learned, nor remove an incorrect belief sitting at the top of its ranked candidates. Setting temperature to 0 (equivalent to always taking argmax) removes randomness in *which* plausible token gets picked when there's genuine ambiguity — but if the single highest-scoring token for a given prompt happens to be a confidently wrong fact, greedy decoding selects exactly that wrong fact with certainty, every time it's asked. Decoding parameters address only the sampling noise layered on top of the model's knowledge; fixing the actual knowledge/confidence gap requires a different intervention — retrieval-grounding (RAG), calibration/RLHF training, or explicit "say you don't know" instruction-tuning. Greedy decoding, if anything, makes outputs *more* consistently wrong on questions the model is confidently mistaken about, not less (**C** has the direction backwards for that specific failure mode).

Q20

LORA INITIALIZATION

In LoRA, matrix B is initialized to all zeros and A to small random values. What would go wrong under (i) both A and B initialized to zero, and (ii) both initialized randomly?

A (i) trains fine, just slowly; (ii) is functionally identical to LoRA's actual scheme

B (i) makes the gradient through BA zero everywhere, so the adapter never starts learning (a dead layer); (ii) starts from a random, nonzero perturbation of a working pretrained model, immediately degrading its behavior before any useful adaptation occurs ✓ CORRECT

C Both alternatives are mathematically equivalent to the actual B=0, A=random scheme

D (i) causes exploding gradients; (ii) causes vanishing gradients

EXPLANATION

LoRA's forward pass is $h = Wx + (\alpha/r)\cdot BAx$, where W is frozen and only B, A are trained. Case (i) — both zero: the gradient with respect to A is proportional to B^T times the upstream gradient; since B=0, this gradient is exactly zero, so A can never move away from zero either — the adapter is a permanently dead branch that never starts learning, not merely a slow learner as **A** suggests. Case (ii) — both random: at step zero, BA is now some random nonzero matrix, so the model's output immediately differs from the base pretrained model's output before any gradient-based correction has occurred — this injects noise into an otherwise-working model, forcing training to first "dig out" of self-inflicted degradation. The actual scheme (B=0, A=random) is the unique choice giving both desired properties simultaneously: BA=0 exactly at initialization (identical to the base model, zero degradation), while a nonzero gradient can still reach A immediately, because $\partial L/\partial A$ depends on the upstream loss gradient and the input x — not on B already being nonzero (only $\partial L/\partial B$ requires A to be nonzero, which it already is by design). Neither alternative is equivalent to this (**C** false), and the actual failure modes are a dead layer and an initial-degradation problem, not exploding/vanishing gradients specifically (**D** false).
